/Users/mery/lectures/libraryframac/project-divers/exercices.tex

Table des matières

1 TD5 **2**

2 TD6 **5**

3 TD7 **6**
3.1 Annotations 6
3.2 Contrat division euclidienne 8
3.3 Contrat factorielle 9

4 TD8 **12**
4.1 Power2 12
4.2 Mutables, pointeurs etc 14

Cours MOdélisation, Vérification et EXpérimentations
Exercices (avec les corrections)
Utilisation d'un environnement de vérification Frama-c (I)
par Dominique Méry
26 avril 2026

1 TD5

Exercice 1 *annotations03.c,annotations04.c*
Nous vous donnons des annotations que vous devez analyser avec Frama-c.

Listing 1 – annotations03.c

Question 1.1
```
#include <limits.h>
/*@ requires a >= 0 && b>= 0;
  @ assigns \nothing;
  @ ensures \result == \old(a)+\old(b)-2;

  @*/
int annotations03(int a,int b)
{
  int x,y,z;
  x = a;
/*@  assert l1: x == a; */
  y = b;
/*@  assert l2: x == a && y == b; */
   z = a+b-2;
/*@  assert l3: x == a && y == b && z==a+b-2; */
  return(z);
}
```

Listing 2 – annotations03c.c

```
gg#include <limits.h>
/*@ requires a >= 0 && b>= 0 && a+b <= INT_MAX; // ajput de a+b <= INT_MAX pour RTE
  @ assigns \nothing;
  @ ensures \result == \old(a)+\old(b)-2;

  @*/
int annotation(int a,int b)
{
  int x,y,z;
  x = a;
/*@  assert l1: x == a; */
  y = b;
/*@  assert l2: x == a && y == b; */
   z = a+b-2;
/*@  assert l3: x == a && y == b && z==a+b-2; */
  return(z);  // \result = z
}
```

Listing 3 – annotations04.c

Question 1.2
```
/*@ requires a >= 0;
  @ assigns \nothing;
```

```
  @ ensures \result == 0;
  @*/
int annotation(int a)
{
  int x;
  x = a;
  return(x);

  }
```

Listing 4 – annotations04c.c

```
/*@ requires a ==  0;
  @ assigns \nothing;
  @ ensures \result == 0;
  @*/
int annotation(int a)
{
  int x;
  x = a;
  return(x);

  }
```

Exercice 2 *annotations05.c*
Soit le petit programme suivant

Listing 5 – annotations05.c

```
void ex(void) {

    int x=2,y=4,z,a;

   //@ assert x <= y;

    x = x*x;
   //@ assert x == a*y && x* 2*x >= 8;
    y = 2*x;
   z = x + y;
   //@ assert z == x+y && x* y >= 8;
}
```

Analyser le correction des annotations avec Frama-c et trouver a pour que cela soit correctement analysé.

◇– **Solution de l'exercice 2**

Listing 6 – annotations05c.c

```
// Utiliser frama-c et demander -wp-status
// en deduire a = 0
void ex(void) {

    int x=2,y=4,z,a=1;

   //@ assert x <= y;
```

```
  x = x*x;
 //@ assert x == a*y && x* 2*x >= 8;
  y = 2*x;
 z = x + y;
 //@ assert z == x+y && x* y >= 8;
}
```

Fin 2

Exercice 3 *Soit le petit programme suivant*

Listing 7 – annotations06.c

```
void ex(void) {
  int x0,y0,z0;
    int x=x0,y=x0,z=x0*x0;
  //@ assert l1: x == y && z == x*y;
  x = x*x;
  //@ assert l2: x == y*y && z == x;
  y = x;
    //@ assert l3:  x + y + 2*z == (x0+x0)*(x0+x0) ;
  z = x + y + 2*z;
 //@ assert z == (x0+x0)*(x0+x0);
}
```

Analyser la correction des annotations avec Frama-c et définir un contrat en supposant que l'on ajoute une instruction return(z).

◇– **Solution de l'exercice 3**

Listing 8 – annotations06c.c

```
/*@  assigns \nothing;
  @ ensures \result ==(x0+x0)*(x0+x0) ;

  @*/
int  ex(int x0,int y0,int z0) {
    int x=x0,y=x0,z=x0*x0;
  //@ assert l1: x == y && z == x*y;
  x = x*x;
  //@ assert l2: x == y*y && z == x;
  y = x;
    //@ assert l3:  x + y + 2*z == (x0+x0)*(x0+x0) ;
  z = x + y + 2*z;
 //@ assert z == (x0+x0)*(x0+x0);
  return(z);
}
```

Fin 3

Exercice 4 *annotation07.c*
Soit le petit programme suivant

Listing 9 – annotations07.c

```
/*@
  assigns \nothing;
   ensures \result >= x && \result >= y && (\result == x || \result == y);
 */

int max ( int x, int y ) {
```

```
  if ( x >=y )
    {
      return x ;
    }
  return y ;
}
```

Analyser la correction des annotations avec Frama-c. Ce programme calcule la valeur du maximum de deux valeurs. Il faudra définir correctement le contrat.

◇– **Solution de l'exercice 4**

Listing 10 – annotations07c.c

```
#include <limits.h>
/*@
  assigns \nothing;
   ensures \result >= x && \result >= y && (\result == x || \result == y);
 */
int max ( int x, int y ) {

  if ( x >=y )
    {
        //@ assert  x>= y;
      return x ;
       //@ assert  x>= y;
    }
        //@ assert  x< y;
  return y ;
        //@ assert  x< y;
}
```

Fin 4

2 TD6

Exercice 5 *La définition structurelle des transformateurs de prédicats est rappelée dans le tableau ci-dessous :*

S	*wp(S)(P)*
X :=E(X,D)	$P[e(x,d)/x]$
SKIP	P
$S_1; S_2$	$wp(S_1)(wp(S_2)(P))$
IF B S_1 ELSE S_2 FI	$(B \Rightarrow wp(S_1)(P)) \wedge (\neg B \Rightarrow wp(S_2)(P))$

- *Axiome d'affectation :* $\{P(e/x)\}$***X :=E(X)***$\{P\}$.
- *Axiome du saut :* $\{P\}$***skip***$\{P\}$.
- *Règle de composition : Si* $\{P\}\boldsymbol{S}_1\{R\}$ *et* $\{R\}\boldsymbol{S}_2\{Q\}$*, alors* $\{P\}\boldsymbol{S}_1\boldsymbol{;}\boldsymbol{S}_2\{Q\}$.
- *Si* $\{P \wedge B\}\boldsymbol{S}_1\{Q\}$ *et* $\{P \wedge \neg B\}\boldsymbol{S}_2\{Q\}$*, alors* $\{P\}$***if B then*** $\boldsymbol{S}_1$ ***then*** $\boldsymbol{S}_2$ ***fi***$\{Q\}$.
- *Si* $\{P \wedge B\}\boldsymbol{S}\{P\}$*, alors* $\{P\}$***while B do S od***$\{P \wedge \neg B\}$.
- *Règle de renforcement / affaiblissement : Si* $P' \Rightarrow P$*,* $\{P\}\boldsymbol{S}\{Q\}$*,* $Q \Rightarrow Q'$*, alors* $\{P'\}\boldsymbol{S}\{Q'\}$.

Question 5.1 *Simplifier les expressions suivantes :*

1. *WP(X :=X+Y+7)(x+y=6)*
2. *WP(X :=X+Y)(x<y)*

Question 5.2 *On rappelle que* $\{P\}\boldsymbol{S}\{Q\}$ *est défini par l'implication* $O \Rightarrow WP(S)(Q)$*. Pour chaque point énuméré ci-dessous, monter que la propriété* $\{P\}\boldsymbol{S}\{Q\}$ *est valide ou pas en utilisant la définition suivante :*

$$\{P\}\boldsymbol{S}\{Q\} = P \Rightarrow WP(S)(Q)$$

1. $\{x{+}y = 7\}$**X :=Y+X**$\{2{\cdot}x{+}y = 6\}$
2. $\{x < y\}$**IF** $\boldsymbol{x \neq y}$ **THEN** ***x := 5*** **ELSE** ***x := 8*** **FI**$\{x \in \{5, 8\}\}$

Question 5.3 *Utiliser frama-c pour vérifier les éléments suivants :*

1. $\{x{+}y = 7\}$**X :=Y+X**$\{2{\cdot}x{+}y = 6\}$
2. $\{x < y\}$**IF** $\boldsymbol{x \neq y}$ **THEN** ***x := 5*** **ELSE** ***x := 8*** **FI**$\{x \in \{5, 8\}\}$

3 TD7

3.1 Annotations

Exercice 6 *Annotations*
On rappelle que l'annotation suivante du listing 11 est correcte , si les conditions suivantes sont vérifiées :

— $pre(v_0) \wedge v = v_0 \Rightarrow A(v_0, v)$
— $pre(v_0) \wedge B(v_0, v) \Rightarrow post(v_0, v)$
— $A(v_0, v) \Rightarrow wp(v = f(v))(B(v_0, v)$ *où* $wp(v = f(v))(B(v_0, v))$ *est définie par* $B(v_0, v)[f(v)/v)]$.

Dans le cas de Frama-c, la valeur initiale d'une variable v est notée $\backslash at(v, Pre)$ *et aussi* $\backslash old(v)$ *. Nous utiliserons la notation* v_0 *dans cet exercice.*

Listing 11 – contrat

```
requires pre(v)
ensures post(\old(v),v)
type1 truc(type2 v)
  /*@ assert A(v0,v); */
  v = f(v);
  /*@  assert  B(v0,v);      */
return  val;
```

Soient les annotations suivantes. Les variables sont supposées de type int.

Question 6.1 *annotations11.c*

$\ell_1 : x = 64 \ \wedge \ y = x{\cdot}z \ \wedge z = 2{\cdot}x$
$Y := X{\cdot}Z$
$\ell_2 : \ y{\cdot}z = 2{\cdot}x{\cdot}x{\cdot}z$

Montrer que l'annotation est correcte ou incorrecte en utilisant Frama-c

Listing 12 – annotations11.c

```
/*@ requires  x0 == 64 && y0 == x0*z0  &&  z0==2*x0;
 */
void  ex61(int x0,int y0,int z0) {
  int x = x0,y=y0,z=z0;
  //@  assert l1: x == 64 && y == x*z &&  z==2*x;
  y = x*z;
  //@  assert l2:  y*z == 2*x*x*z;
}
```

Listing 13 – annotation11bis.c

```
/*@
  requires x0==64 && y0==x0*z0 && z0==2*x0;
```

```
  ensures \result == 0;
*/
int ex(int x0,int y0,int z0) {
  int x=x0,y=y0,z=z0;

  //@ assert x==64 && y==x*z && z==2*x;
  y = x*z;
  //@ assert y*z == 2*x*x*z;
  return 0;


}
```

Question 6.2

$\ell_1 : x = 10 \ \wedge \ y = z{+}x \ \wedge z = 2{\cdot}x$
$y := z{+}x$
$\ell_2 : x = 10 \ \wedge \ y = x{+}2{\cdot}10$

Montrer que l'annotation est correcte ou incorrecte en utilisant Frama-c

Listing 14 – annotations12.c

```
/*@ requires x0 == 10 && y0 == z0+x0 && z0==2*x0;
 */
void ex81(int x0,int y0,int z0) {
  int x = x0,y=y0,z=z0;
  //@ assert x == 10 && y == z+x&& z==2*x;
  y = z+x;
  //@ assert x== 10 && y == x+2*10;
}
```

Exercice 7 *annotations13.c*
Soit le contrat suivant :

variables X, Y, Z
requires $x_0 >= 0 \wedge y_0 >= 0 \wedge z_0 >= 0 \wedge z_0 = 25 \wedge y_0 = x_0{+}1$
ensures $z_f = 100$;
begin
$0 : x^2{+}y^2 = z \wedge z = 25$;
$(X, Y, Z) := (X{+}3, Y{+}4, Z{+}75)$;
$1 : x^2{+}y^2 = z$;
end

Question 7.1 *Traduire ce contrat avec le langage PlusCal et proposer une validation pour que ce contrat soit valide.*

Question 7.2 *Traduire ce contrat en ACSL et vérifier qu'il est valide ou non. S'il est non valide, proposer une correction de la pré-condition et/ou de la postcondition.*

Listing 15 – annotations13.c

```
    /*@
  requires x0>=0 && y0 >= 0 && z0 >= 0 && z0 == 25 && y0==x0+1 && x0*x0+y0*y0==z0;
  ensures \result == 100;
*/
```

```
int f(int x0,int y0,int z0) {
  int x = x0;
  int y = y0;
    int z = z0;
    /*@ assert  x*x + y*y == z &&    z == 25 ;*/
   x = x +3;
y = y +4;
z = z + 75;
/*@ assert  x*x + y*y == z ; */
  return z;
}
```

Listing 16 – annotations13bis.c

```
    /*@
  requires x0>=0 && y0 >= 0 && z0 >= 0 && z0 == 25 &&  y0==x0+1 &&
 x0*x0 + y0*y0 == z0 &&    z0 == 25  &&  (x0+3)*(x0+3) + (y0+4)*(y0+4) == z0+75 ;
  ensures \result == 100;
*/
int f(int x0,int y0,int z0) {
  int x = x0;
  int y = y0;
    int z = z0;
/*@ assert  x*x + y*y == z &&    z == 25 ;*/
/*@ assert  (x+3)*(x+3) + (y+4)*(y+4) == z+75 ; */
   x = x +3;
/*@ assert  x*x + (y+4)*(y+4) == z+75 ; */
y = y +4;
/*@ assert  x*x + y*y == z+75 ; */
z = z + 75;
/*@ assert  x*x + y*y == z ; */
  return z;
}
```

3.2 Contrat division euclidienne

Exercice 8 *division.c, division.h*
Ecrire un contrat pour la division euclidienne en tenat compte de différents cas du dénominateur.

Listing 17 – division.c

```
struct s  division(int a, int b)
{  int rr = a;
   int qq = 0;
   /*@ loop invariant  b <= rr &  ;
        lopp assigns rr,qq;
    while (rr >= b) { rr = rr - b; qq=qq+1;};
   resu.q= qq;
   resu.r = rr;
  return resu;
}
}
```

Listing 18 – division.h

```
#ifndef _A_H
#define _A_H
#include "structures.h"
/*@ requires a >= 0 && b >= 0;
 @ requires b != 0;
  @ ensures 0 <= \result.r;
  @ ensures \result.r < b;
  @ ensures  a ==  b * \result.q + \result.r;
*/
struct s  division(int a, int b);
#endif
```

Listing 19 – division.c

```
#include <stdio.h>
#include <stdlib.h>
#include "division.h"

struct s  division(int a, int b)
{  int rr = a;
   int qq = 0;
   struct s  silly = {-1,-1};
   struct s resu;
   if (b == 0) {
     return silly;
   }
   else
     {
  /*@
    loop invariant
    ( a ==  b*qq + rr) &&
    rr >= 0;
    loop assigns rr,qq;
    loop variant rr;
   */
   while (rr >= b) { rr = rr - b; qq=qq+1;};
   resu.q= qq;
   resu.r = rr;
  return resu;
}
}
```

3.3 Contrat factorielle

Exercice 9 *fact.c*
Soit la fonction fact qui calcule la fonction factorielle.

Listing 20 – fact.c

```
int codefact(int n) {
  int y = 1;
  int x = n;
   while (x != 1) {
    y = y * x;
    x = x - 1;
  };
```

```
  return y;

}
```

Définir un contrat pour cette fonction en définissant la fonction mathématique factorielle. Annoter le programme en définissant un invariant de boucle.

Listing 21 – factorial.c

```
#include "factorial.h"

int codefact(int n) {
  int y = 1;
  int x = n;
  /*@ loop invariant x >= 1 && x <= n && mathfact(n) == y * mathfact(x);
    loop assigns x, y;
    loop variant x;
   */
  while (x != 1) {
    y = y * x;
    x = x - 1;
  };
  return y;

}
```

Listing 22 – factorial.h

```
#ifndef _A_H
#define _A_H
/*@ axiomatic mathfact {
  @ logic integer mathfact(integer n);
  @ axiom mathfact_1: mathfact(1) == 1;
  @ axiom mathfact_rec: \forall integer n; n > 1
  ==> mathfact(n) == n * mathfact(n-1);
  @ } */

/*@ requires n > 0;
  decreases n;
  ensures \result == mathfact(n);
  assigns \nothing;
*/
int codefact(int n);
#endif
```

Exercice 10 *inv1.c*
Nous étudions ce petit algorithme qui calcule quelque chose et nous avons exécuté cet algorithme de 0 et 10 pour obtenir la suite suivante :

```
0 --> 0,1 --> 1,2 --> 3,3 --> 7,4 --> 15,5 --> 31,6 --> 63,7 --> 127,8 --> 255,
```

Listing 23 – mathinv1.h

```
#ifndef _A_H
#define _A_H
// Definition of the  mathematical function mathpower2
/*@ axiomatic mathpower {
  @ logic integer mathpower(integer n,integer m);
```

```
  @ axiom mathpower_0: \forall integer n; n >= 0 ==> mathpower(n,0) == 1;
  @ axiom mathpower_in: \forall integer n,m; n >= 0 && m >= 0
  ==> mathpower(n,m+1) == mathpower(n,m)*n;
  @ } */


int inv1(int x);
#endif
```

Listing 24 – inv1.c

```
#include <limits.h>
#include <mathinv1.h>
/*@ requires x >=0;
    assigns \nothing;
    ensures \result ==  mathpower(2,x)-1;
*/
int inv1(int x)
{ int u=0;
  int k=0;
  /*@ loop invariant 0<= k && k <= x;
    @  loop invariant u == mathpower(2,k)-1;
     @ loop assigns u,k;
     @loop variant x-k;
  */
  while (k  < x)
    { u=2*u+1;
      k=k+1;
    };
  return(u);
    }
```

Si on utilise la fonction power2, on obtient la suite suivante :

```
0 --> 1,1 --> 2,2 --> 4,3 --> 8,4 --> 16,5 --> 32,6 --> 64,7 --> 128,8 --> 256,9 --
```

Question 10.1 *On comprend que l'algorithme calcule la suite u_n d'entiers telle que $\forall n \in \mathbb{N}$: $u_n = 2^n - 1$. En particulier, $u_0 = 0$.*
Donner une définition de u_{n+1} en fonction de u_n en calculant le rapport $\frac{u_{n+1}+1}{u_n+1}$

Question 10.2 *Ecrire un contrat pour cet algorithme, en précisant la clause requires et la clause ensures.*

Question 10.3 *Proposer un invariant de boucle en vous aidant de la suite u_n et monter qu'il est correct pour cette preuve de correction.*

Question 10.4 *Exprimer la terminaison de cet algorithme et justifier qu'il termine pour la précondition choisie.*

Listing 25 – inv1.c

```
#include <stdio.h>
#include <limits.h>
#include <math.h>

// Definition of the  mathematical function mathpower2
/*@ axiomatic mathpower {
```

```
  @ logic integer mathpower(integer n,integer m);
  @ axiom mathpower_0: \forall integer n; n >= 0 ==> mathpower(n,0) == 1;
  @ axiom mathpower_in: \forall integer n,m; n >= 0 && m >= 0
  ==> mathpower(n,m+1) == mathpower(n,m)*n;
  @ } */

/*@  requires  0 <= x;
     assigns \nothing;
     ensures \result + 1 == mathpower(2,x);
*/

int inv1(int x)
{ int u=0;
  int k=0;
      /*@ loop invariant u + 1  == mathpower(2,k);
         @ loop invariant 0 <= k && k  <= x;
         @ loop assigns k,u;
          @ loop variant x-k; */

  while (k  < x)
    { u=2*u+1;
      k=k+1;
    };
  return(u);
    }
```

4 TD8

4.1 Power2

Exercice 11 *power2.c*

Listing 26 – qpower2.c

```
// #include <limits.h>
/*@ axiomatic auxmath {
  @ axiom  rule1: \forall int n; n >0 ==> n*n == (n-1)*(n-1)+2*n+1;
  @ } */

/*@  requires  0 <= x;
     ensures \result == x*x;
*/
int power2(int x)
{int  r,k,cv,cw,or,ok,ocv,ocw;
  r=0;k=0;cv=0;cw=0;or=0;ok=k;ocv=cv;ocw=cw;
      /*@ loop invariant cv == k*k;
         @ loop invariant k  <= x;
         @ loop invariant cw  == 2*k;
         @ loop invariant  4*cv  == cw*cw;
         @ loop assigns k,cv,cw,or,ok,ocv,ocw; */
  while (k<x)
        {
          ok=k;ocv=cv;ocw=cw;
          k=ok+1;
          cv=ocv+ocw+1;
          cw=ocw+2;
```

```
        }
  r=cv;
  return(r);
}
```

Soit le fichier `qpower2.c` *qui est patriellement complété et qui permet de calculer le carré d'un nombre naturel. Léxercice vise à compléter les points qui manquent puis de simplifier le résultat et de montrer l'équivalence de deux fonctions.*

Question 11.1 *Compléter le fichier* `qpower2.c` *et produire le fichier* `power2.c` *qui est vérifié avec Frama-c.*

Question 11.2 *Simplifier la fonction itérative en supprimant les variables commençant par la lettre* `o`*. Puis vérifier les fonctions obtenues avec frama-c.*

Listing 27 – qmainpower2.c

```
#include <stdio.h>
#include <math.h>

int power2(int x)
{int  r,k,cv,cw,or,ok,ocv,ocw;
  r=0;k=0;cv=0;cw=0;or=0;ok=k;ocv=cv;ocw=cw;
  while (k<x)
        {
          ok=k;ocv=cv;ocw=cw;
          k=ok+1;
          cv=ocv+ocw+1;
          cw=ocw+2;

        }
  r=cv;
  return(r);
}

int p(int x)
{
  int r;
  if (x==0)
        {
          r=0;

        }
  else
        {
          r= p(x-1)+2*x-1;

        }
  return(r);
}

int check(int n){
  int r1,r2,r;
  r1 = power2(n);
  r2 = p(n);
  if (?? == ??)
```

```
    { r = ??;
    }
  else
    { r = ??;
    };
  return r;
 }
int main()
{
  int val1,val2,val3,num;
   printf("Enter␣a␣number:␣");
   scanf("%d", &num);
   val1 = power2(num);
     val2 = p(num);
     val3 = check(num);
     printf("Et␣le␣rÃ©sultat␣␣pour␣n=␣%d:␣%d␣%d␣%d\n", num, val1,val2,val3);
   return 0;
}
```

Question 11.3 *Compléter les contrats*

Question 11.4 *En fait, vous avez montré que les deux fonctions étaient équivalentes. Expliquez pourquoi en quelques lignes.*

4.2 Mutables, pointeurs etc

Listing 28 – qpierre1.c

Exercice 12

```
//@ admit p == q;
*p = 1;
//@ assert *p + *q == 2;
```

Question 12.1 *Does Frama-C prove the above snippet ?*

Question 12.2 *Recall the weakest precondition for an assignment.*

Question 12.3 *Compute the subsequent verification condition for the snippet.*

Question 12.4 *Is the subsequent verification condition verified ? If not, what to change ?*

Listing 29 – pierre1.c

```
void main()
{
    //@ assert 1 + 1 == 2;
  int *p;
  int *q;
  //@ assert 1 + 1 == 2;
   //@ assert Mint[p|->1](p) + Mint[p|->1](p) == 2;
  //@ admit p == q
;  //@ assert Mint[p|->1](p) + Mint[p|->1](q) == 2;
  Mubt = Mint[p|-> 1];
  *p = 1;
 //@ assert Mint[p] + Mint[q] == 2;
//@ assert *p + *q == 2;
}
```

Listing 30 – apierre1.c

```
void main()
{
  int *p;
  int *q;
  //@ admit p == q;
*p = 1;
//@ assert *p + *q == 2;
}
```

Exercice 13 *Here is the basic swap function (no xor trick here) :*

Listing 31 – pierre2.c

```
/*@
  requires \valid(a) && \valid(b);
  assigns *a,*b;
  ensures *a == \old(*b);
  ensures *b == \old(*a);
 */
void swap(int *a, int *b){
int tmp = *a;
*a = *b;
*b = tmp;
}
```

Question 13.1 *Write the expected post-condition. Check it.*

Question 13.2 *As warned by Frama-C, check run-time errors. Write the weakest pre- condition to pass.*

Question 13.3 *As warned by Frama-C, specify the weakest side-effects to pass.*

Question 13.4 *Also perform smoke tests.*

Exercice 14 *REVERSE Write, specify and prove the function* `void reverse(int*array, size_t len)` *that reverses a vector in place. Do use the previously proved swap function. Hint : Take care of the unmodified part of the vector at some iteration of the loop.*

Listing 32 – pierre2.c

```
#include <limits.h>

/*@
   requires  \valid(a) && \valid(b) ;
   assigns *a;
   assigns *b;
   ensures A: *a == \old(*b);
    ensures B: *b == \old(*a);
  @*/
void swap(int* a, int* b) {
  int tmp = *a;
  *a = *b;
  *b = tmp;
}
```

```
/*@
predicate
Unchanged{K,L}(int* a, integer first, integer last) =
\forall integer i; first <= i < last ==> \at(a[i],K) == \at(a[i],L);
predicate
Unchanged{K,L}(int* a, integer n) = Unchanged{K,L}(a, 0, n);
*/
```

```
/*@
predicate
Reverse{A,B}(int* a, integer n, int* b, integer l, integer r) =
\forall integer k; l <= k < r ==> \at(a[k], A) == \at(b[n-1-k], B);
predicate
Reverse{A,B}(int* a, integer n, int* b) =
Reverse{A,B}(a, n, b, 0, n);
predicate
Reverse{A,B}(int* a, integer n) = Reverse{A,B}(a, n, a);
*/
```

```
/*@
  requires n >0;
  requires valid: \valid(a + (0..n-1));
  assigns a[0..(n-1)];
  ensures reverse: Reverse{Here,Old}(a, n);
*/
void reverse(int* a, int n)
{
  const int half = n / 2;
/*@
loop invariant bound: 0 <= i <= half;
loop invariant left: Reverse{Here,Pre}(a, n, a, 0, i);
loop invariant middle: Unchanged{Here,Pre}(a, i, n-i);
loop invariant right: Reverse{Here,Pre}(a, n, a, n-i, n);
loop assigns i, a[0..n-1];
loop variant half - i;
*/
for (int i = 0; i < half; ++i) {
swap(&a[i], &a[n - 1 - i]);
 };
}
```

Exercice 15 *COPY*

Question 15.1 *Write, specify and prove the function* `void copy(int const *src, int *dst, size_t` *that copies a range of values into another array, start-*
ing from the first cell of the array. First consider (and specify) that the two ranges are entirely separated.

Question 15.2 *In fact, such a strong separation is not necessary. Weaken the previous one while still meeting the postcondition.*

Listing 33 – pierre2.c

```
/*@
predicate
EqualRanges{A,B}(int* a, integer m, integer n, int* b) =
\forall integer i; m <= i < n ==> \at(a[i], A) == \at(b[i], B);
predicate
EqualRanges{A,B}(int* a, integer n,int* b) =
EqualRanges{A,B}(a, 0, n, b);
*/

/*@
predicate
Unchanged{K,L}(int* a, integer first, integer last) =
\forall integer i; first <= i < last ==> \at(a[i],K) == \at(a[i],L);
predicate
Unchanged{K,L}(int* a, integer n) = Unchanged{K,L}(a, 0, n);
*/

/*@
requires n >0;
requires valid: \valid_read(a + (0..n-1));
requires valid: \valid(b + (0..n-1));
requires sep: \separated(a + (0..n-1), b);
assigns b[0..n-1];
ensures equal: EqualRanges{Old,Here}(a, n, b);
*/
void copy(const int* a, const int n, int* b)
{
/*@
loop invariant bound: 0 <= i <= n;
loop invariant equal: EqualRanges{Pre,Here}(a, i, b);
loop invariant equal: Unchanged{Pre,Here}(a, i, n);
loop assigns i, b[0..n-1];
loop variant n-i;
*/
for (int i = 0; i < n; ++i) {
b[i] = a[i];
}
}
```